Laboration 2: Cellular Automata

Group Supermodels
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

May 5, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Game of Life **3**
- 2.1 Game of Life as a discrete dynamical system . . . 3
- 2.2 Encoding of Inputs . . . 4
- 2.3 Reading outputs . . . 4
  - 2.3.1 Observation Window, Detection logic & Synchronization . . . 4
- 2.4 Numerical methods for Game of Life . . . 5
  - 2.4.1 Computational domain . . . 5
  - 2.4.2 Initial conditions & Time evolution . . . 5
  - 2.4.3 Logic gate engineering . . . 5
  - 2.4.4 Execution . . . 5
- 2.5 Simulation Results . . . 6
  - 2.5.1 NOT-gate realisation . . . 6
  - 2.5.2 AND-gate realisation . . . 7
  - 2.5.3 OR-gate realisation . . . 9

3 Neuron Cellular Automaton **11**
- 3.1 Neuron Automaton Theory . . . 11
- 3.2 Neuron Cellular Automaton . . . 11
- 3.3 Neuron Automaton Results . . . 12

4 Greenberg-Hastings Cellular Automaton **14**
- 4.1 GHCA theory . . . 14
- 4.2 GHCA Method . . . 14
- 4.3 GHCA Results . . . 14

5 Interpretation **15**

6 Conclusion **16**

7 Use of LLM tools **17**
- 7.1 Game of Life . . . 17
- 7.2 Neuron Cellular Automaton . . . 17

1. Problem statement

In this lab we will be investigating classical examples of cellular automata. Cellular automata are ...

The goal of the lab is to ...

All of these tasks will give us an overview of the characteristics in cellular automata ...

2. Game of Life

2.1. Game of Life as a discrete dynamical system

Conway's Game of Life can be viewed as a discrete dynamical system on a rectangular lattice $X^{(t)} \in \{0, 1\}^{m \times n}$, where each entry $X_{ij}^{(t)}$ represents the state of the cell at site (i, j) at time t. The evolution of the system is determined by update rules that specify the state of each cell at the next time step, $X_{ij}^{(t+1)}$.

Definition 2.1 – Conway's rules [2]

1. **Survival**: a live cell stays alive if it has 2 or 3 live neighbors.
2. **Birth**: a dead cell becomes alive if it has exactly 3 live neighbors.
3. **Death by isolation**: a live cell dies if it has fewer than 2 live neighbors.
4. **Death by overcrowding**: a live cell dies if it has more than 3 live neighbors.
5. **Neighborhood**: the neighbors of a cell are the 8 surrounding cells.

Let $N_{ij}^{(t)}$ denote the degree of node $X_{ij}^{(t)}$ at time t. Since each cell has eight neighbors, the relevant directions are the cardinal and intercardinal directions. Self-interaction is excluded, so the evolution of a cell depends only on its neighborhood. The quantity $N_{ij}^{(t)}$ can be computed as

$$N_{ij}^{(t)} = \sum_{p=-1}^{1} \sum_{q=-1}^{1} K_{p+2,q+2} X_{i+p,j+q}^{(t)}, \tag{2.1}$$

where K is a 3×3 kernel with ones except the center entry that is equal to zero. Equivalently $N^{(t)} = K * X^{(t)}$, where $K * X^{(t)}$ denotes the convolution of the lattice $X^{(t)}$ with the kernel K.

The update rule can then be written as

$$X_{ij}^{(t+1)} = \begin{cases} 1, & \text{if } X_{ij}^{(t)} = 1 \text{ and } N_{ij}^{(t)} \in \{2, 3\}, \\ 1, & \text{if } X_{ij}^{(t)} = 0 \text{ and } N_{ij}^{(t)} = 3, \\ 0, & \text{otherwise.} \end{cases} \tag{2.2}$$

This shows that the update rule is a non-linear composition of a linear map:

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) + X^{(t)} \odot \mathbb{1}\left(N^{(t)} = 2\right), \tag{2.3}$$

We use dirichlet boundary conditons or zero padding to handle the adjacency of boundary nodes.

Logic gates in the Game of Life

In classical computation, the basic logic gates `AND`, `OR`, and `NOT` act on one or two binary input streams and produce a binary output stream. They implement a local transformation of information.

In the Game of Life, by contrast, computation is realized not by abstract Boolean operators but by spatially embedded dynamical processes on a lattice. The initial condition is a geometric configuration, and Conway's update rule evolves this configuration through time. The problem of building a logic gate is therefore a problem of geometric design and synchronization: one seeks an initial configuration $X^{(0)}$ that evolves, under the Game of Life rules, into a deterministic spatiotemporal process implementing `AND`, `OR`, or `NOT`.

This is achieved by using gliders as moving signals and glider guns as periodic sources of such signals [3, 4]. Glider trajectories function as wires, constraining the direction of information

flow; in the absence of interference, the signal propagates persistently along these paths. To represent binary information, the presence of a glider stream is interpreted as `1`, while its absence is interpreted as `0`. Eaters are used as stabilizing and filtering structures, suppressing selected streams and enabling controlled interactions between multiple inputs. By translating and positioning guns, eaters, and collision regions appropriately, one obtains deterministic and composable gate constructions [1], so that the output of one gate may be routed as the input of another without loss of logical meaning. computational fidelity.

2.2. Encoding of Inputs

Definition 2.2 – Encoding binary signals

Signal generation is achieved using the Gosper glider gun, which produces a periodic sequence of gliders with fixed phase and spacing. Through geometric transformations (rotations and reflections), identical guns can be oriented to produce signals in any required intercardinal direction.

1. **Signal 1** (`TRUE`): This is represented by the **prescence** of a **glider**. This signal propagates dynamically across a spatial lane across the lattice in fixed intercardinal directions.
2. **Signal 0** (`FALSE`): Defined as the **absence** of a **glider**. Its realization depends on the observation point. There are two basic cases:
   - **Blocking**: Physically *obstructing* a Gosper gun.
   - **Annihilation**: Using collision trajectories or *eaters* to remove a glider from the stream

Definition 2.3 – Signal Interaction Mechanics

Logic is performed through the interaction of these signals. Because gliders are sensitive to **synchronization**, the timing of these interactions is as important as their location.

1. **Glider-Glider Collision**: These collisions are different depending on their spatiotemporal initial configuration.
   - Requires: Antiparallel streams & precise phase sync.
   - Result: **Mutual Annihilation** (functioning as a switch) *or* **Fragmentation**. It also induces a space-dependent signal delay.
2. **Glider-Eater Interaction**: These interactions are *less* sensitive to synchronization.
   - Requires: Specific placement/orientation of the Eater.
   - Result: **Glider Annihilation/Absorpbtion** or **fragmentation**.

2.3. Reading outputs

Reading the state of a logic gate in the Game of Life is a measurement of **spatial-temporal presence**. The output of a gate is not solely a boolean value, but a **vector**. Since the gliders only propagate along intercardinal diagonals (NE, NW, SE, SW), the observation window must be placed precisely in the *firing lane* of the logic gate's exit path.

2.3.1. Observation Window, Detection logic & Synchronization

To detect a 1 signal (the presence of a glider), the observer must define a static bounding box of 4×4 cells at a specific coordinate (x, y) on the lattice. Because a glider is a dynamic pattern that fits within 4×4 area during its four-generation cycle. The output is considered to have the assigned `TRUE` value if the correct number of live cells within the 4×4, and the distribution

are characteristic of the glider. If the observation window is checked at generation T, but the glider arrives at $T + 1$, the `READ` might detect a partial pattern (fragmentation) or an empty box, resulting in a false `0` or instabilities.

2.4. Numerical methods for Game of Life

2.4.1. Computational domain

The computational domain is represented as a two-dimensional rectangular lattice $X \in \{0, 1\}^{n \times n}$, where each entry corresponds to one cell of the automaton.

2.4.2. Initial conditions & Time evolution

Initial conditions are generated programmatically in `GoLogic.py`. The basic building blocks are the Gosper glider gun and the eater, both stored in Run Length Encoding (RLE). A custom parser converts each RLE string into a binary array using `RLE.py` and `add_pattern`. Before adding a pattern one can transform it appropriately with the `transform_pattern` function in `pattern_processing.py`. handle insertion and transformation before simulation. In addition, the function `load_live_cells(filename, n)` loads external initial configurations from text files containing one live cell per line in the format `(x,y)`. The file is converted into an $n \times n$ binary array by setting `grid[y,x] = 1`. This supports the coordinate-list representation used for the initial gate configurations.

The time evolution is implemented as a composition of a linear convolution operation and a point-wise non-linear update. The discrete convolution is computed using `scipy.ndimage.convolve`. A fixed 3×3 kernel is used to compute the Moore neighbourhood sum $N^{(t)} = K * X^{(t)}$. The central entry of the kernel is zero, so the cell itself is excluded from the neighbour count. The update function allows both zero-padding boundaries, set by the parameter `mode="constant"`.

2.4.3. Logic gate engineering

The logical gates `AND`, `OR`, and `NOT` are implemented by spatial arrangements of glider guns and eaters. Signals are encoded as glider streams emitted by Gosper guns. Boolean inputs are represented through the initial configuration: an input value of 1 leaves the corresponding signal unobstructed, whereas an input value of 0 places an eater on the signal path and suppresses propagation. For the `AND` and `OR` gates, all four input combinations were simulated. While the `NOT` gate was tested for both possible input values. The output was read from a designated output lane: the presence of a propagating glider stream was interpreted as logical 1, and its absence as logical 0.

2.4.4. Execution

The implementation can be executed with `python3 GameOfLife.py AND.txt`, adapting the text file for whatever is desired. For a different output, where no file is used, one can simply run the code as `python3 GameOfLife.py`.

2.5. Simulation Results

2.5.1. NOT-gate realisation

Initial conditions

NOT–gate, $t = 0$

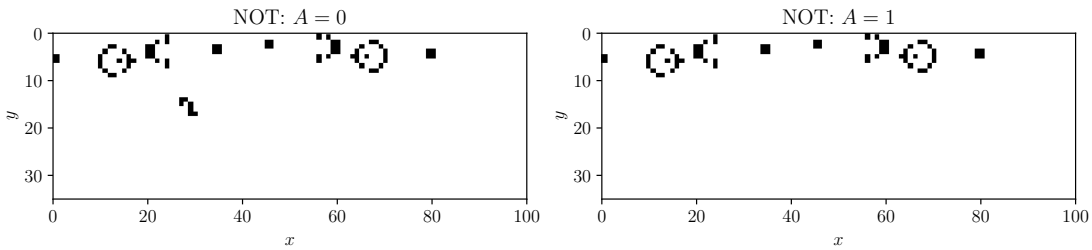

Figure 1. Two Gosper glider guns: the left gun provides a constant input stream, while the input channel injects gliders traveling in the south-east direction. No input is encoded by inserting an eater in the SE input stream. The output is read as gliders propagating in the south-west direction.

Transient state

NOT–gate, $t = 488$

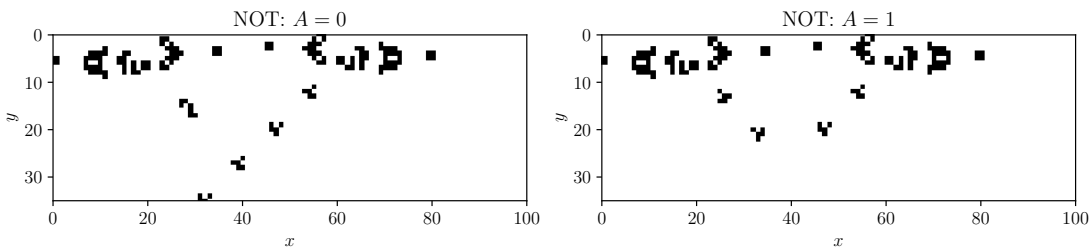

Figure 2. One input channel, A. The input glider moves diagonally from the top to the lower-right part of the grid. The output is observed after the interaction region, in the outgoing diagonal channel. The construction is arranged so that when $A = 0$ an output glider survives, while when $A = 1$ the input glider destroys or blocks the outgoing signal.

2.5.2. AND-gate realisation

Initial conditions

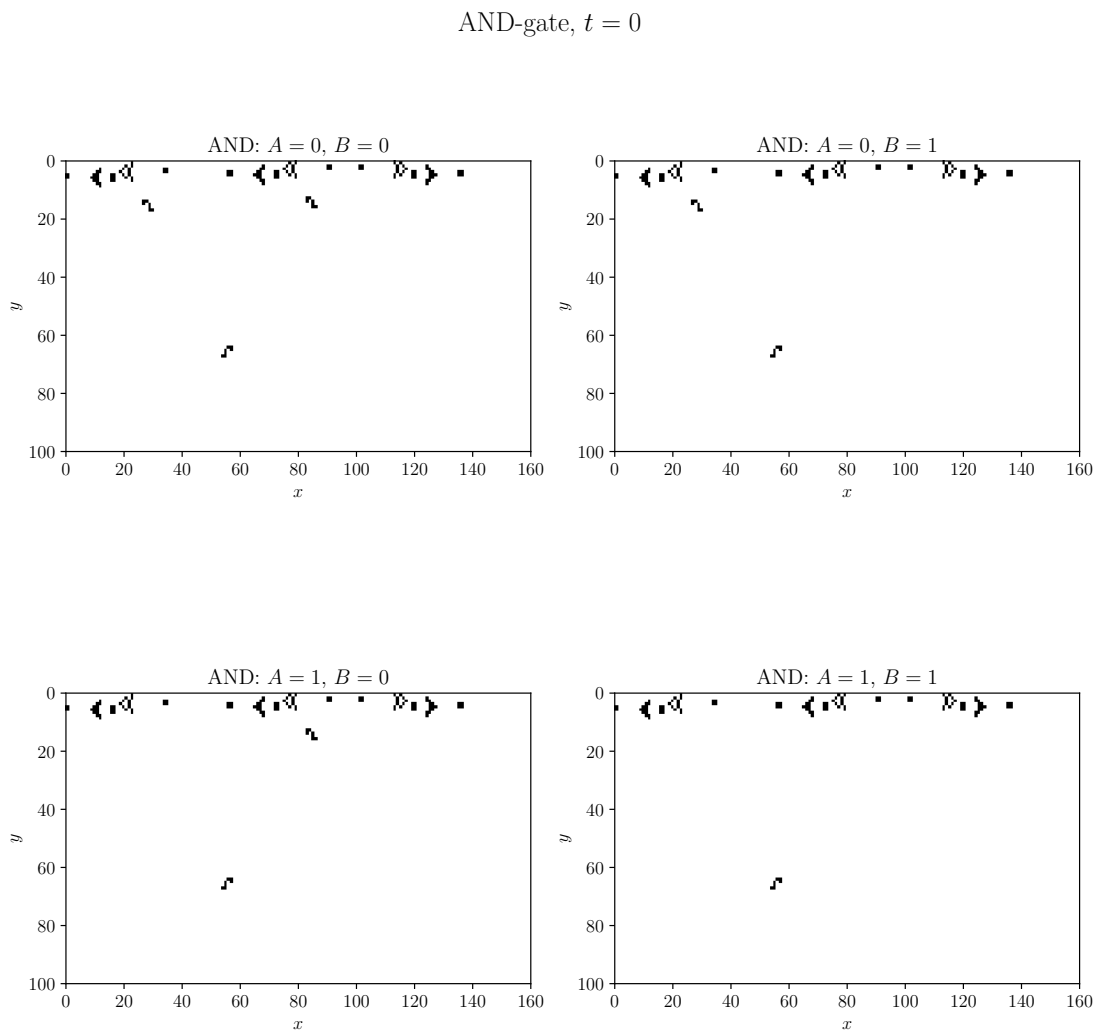

Figure 3. Three Gosper glider guns: the two leftmost generate south-east-directed input streams. An eater suppresses the south-west output unless both inputs are present. In the absence of inputs, additional eaters are placed along the collision paths to block spurious gliders. The output is read as a south-east-directed stream.

94 **Transient state**

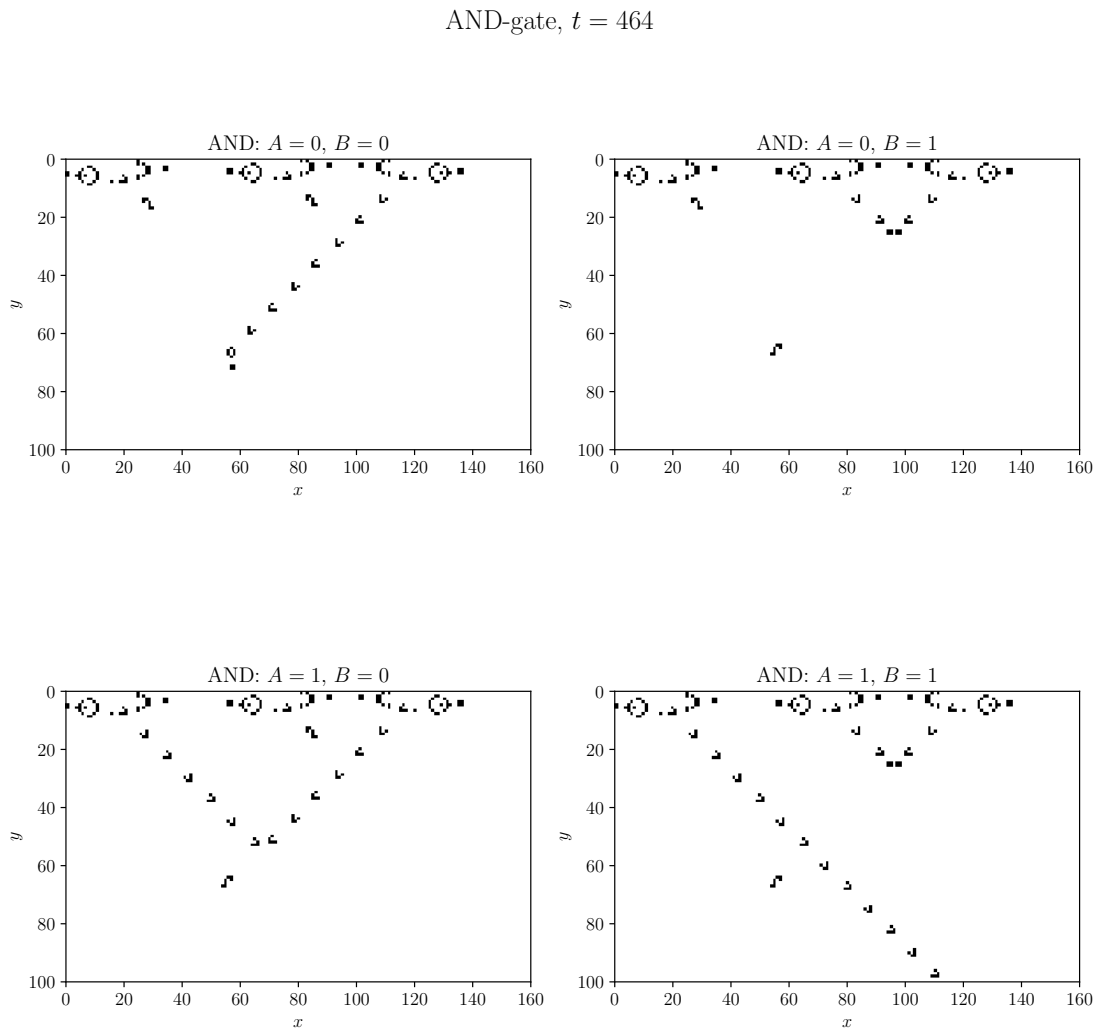

Figure 4. Two input channels are used, A and B. The input gliders move diagonally from the top of the grid towards the lower-right part of the grid, where they interact with the gate structure. The output is observed in the outgoing diagonal channel after this interaction. A glider appears in the output channel only when both input gliders are present, so the output is 1 only for $A = 1$ and $B = 1$.

2.5.3. OR-gate realisation

Initial conditions

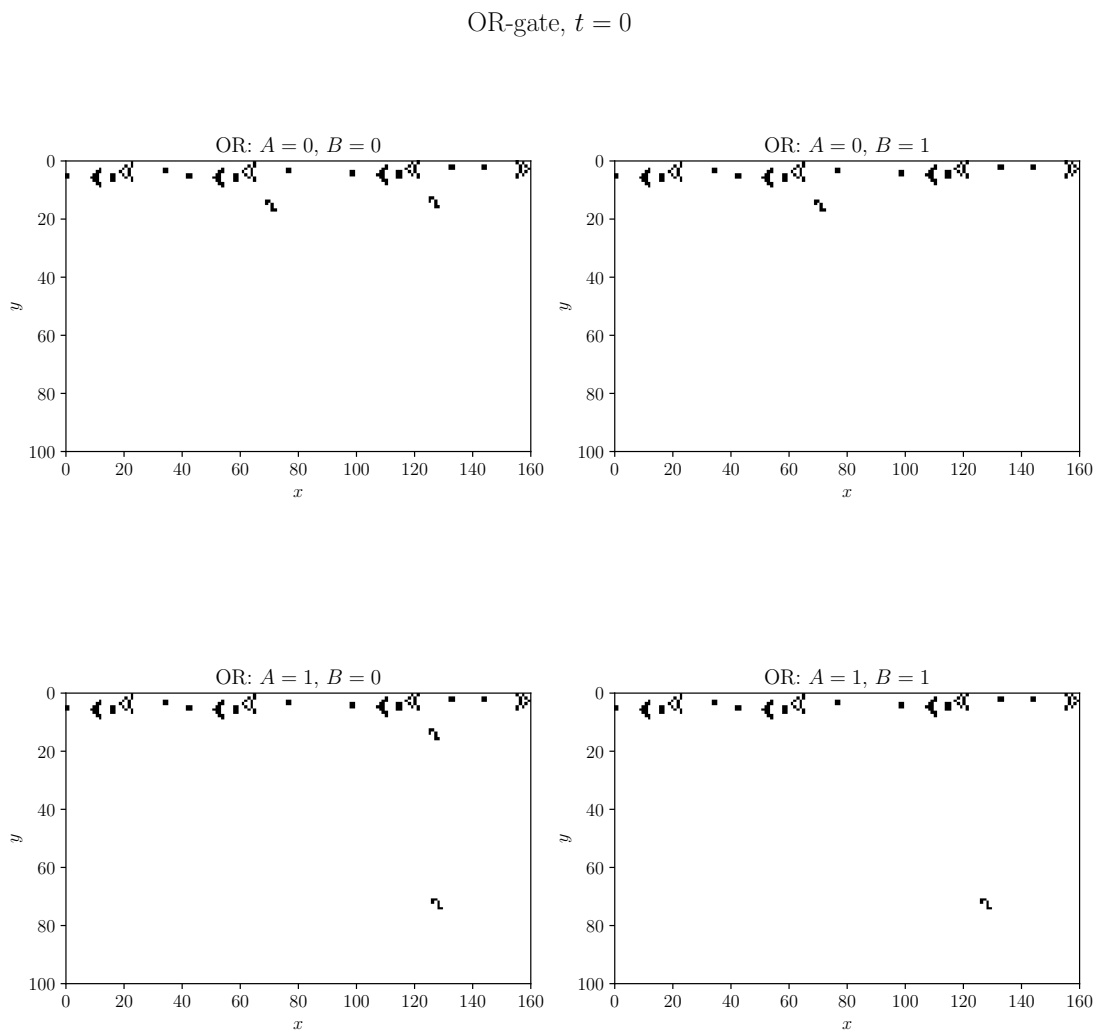

Figure 5. Four Gosper glider guns: the second and third generate south-east-directed input streams. Logical zeros are implemented by placing eaters along the input collision paths. The output is read as a south-east-directed stream.

97 **Transient state**

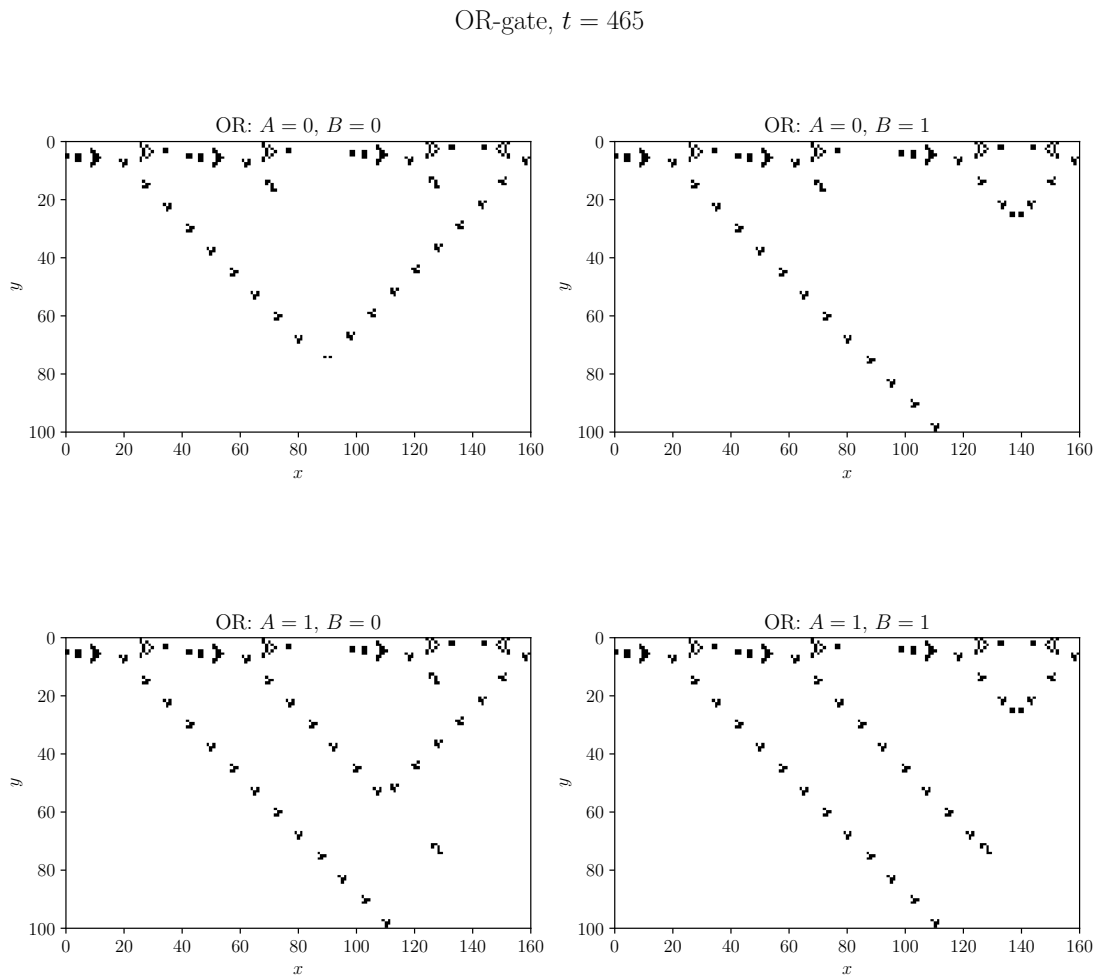

Figure 6. Input gliders also move diagonally from the top of the grid towards the lower-right part of the grid. The output is shown in the outgoing diagonal channel after the region of collision. In this case, the output channel contains a glider when, at least, one of the two input gliders is present.

3. Neuron Cellular Automaton

3.1. Neuron Automaton Theory

Similar to the Game of Life, the neuron automaton is a two-dimensional cellular automaton. Each cell can be in one of three states: ”ready”, ”resting”, and ”firing”. The state of each cell at the next time step is determined by its current state of the cell and the states of its eight neighbors according to the following rules:

Definition 3.1 – Neuron cellular automata rules

1. **Firing**: a cell in the ”ready” state will transition to the ”firing” state in the next time step if it has at least two neighbors in the ”firing” state.
2. **Resting**: a cell in the ”firing” state will transition to the ”resting” state in the next time step.
3. **Ready**: a cell in the ”resting” state will transition to the ”ready” state in the next time step.

This kind of cellural automata simulates the behavior of neurons in a simplified way. The ready to firing to resting cycle mimics the action potential of a neuron, where a neuron fires when it receives enough stimulation from its neighbors and then goes through a refractory period before it can fire again. The interactions between cells can lead to complex patterns, including waves of activity and oscillations, which are reminiscent of neural activity in the brain.

3.2. Neuron Cellular Automaton

The script for the implementation of the neuron cellular automaton is `neuron.py`. The code is based on `gameOfLifeInteractiveToric.py`, which was available on the Studium page of the course. The code can be run using command `python3 neuron.py`.

Now, we will present how the code works. At first, the number of rows and columns of the grid as well the speed of the simulation are defined. Additionally, the rules text is added. This text is displayed when clicking the ”rules”-button while running the script. Next, we have implemented the new function `count_neighbors_by_state` that can calculate the number of neighbours in a certain state. This function is necessary as we will be working with three different states instead of two. The function takes the grid and the state that we want to determine the number of neighbours for as arguments. The neighbours for each gridpoint are found using `numpy.roll` to include the periodic boundary conditions. As the next step we have updated the `lifeStep` function. Now, we first create the grids of neighbours in states 1 and 2 and apply the rules afterwards. State 0 represents the resting neurons (white), state 1 the ready neurons (black) and state 2 the firing neurons (red). The function then returns the updated grid.

In the constructor of the `GameOfLifeApp` class the visualisation is defined. This includes the buttons that are displayed and their actions. The time step for the continuous run is defined as well.

The following parameters are fixed in the code: the number of rows and columns is set to 30 each. For the random population of the grid, the probability for state 0 is 0.1, for state 1 the probability is 0.8 and 0.1 for state 2. Initially, all cells are in state 1 (ready). Clicking once sets them to state 2 (firing) and clicking another time to state 0 (resting).

To use our simulation of the neuron cellular automaton run `neuron.py`.

3.3. Neuron Automaton Results

Using random initial conditions in the neuron automation usually led to either the whole system eventually dying out and being inert in the ”ready” condition or to stable, infinitely forward moving gliders appearing. An example of a random initial pattern evolving into stable gliders is shown in figure 7.

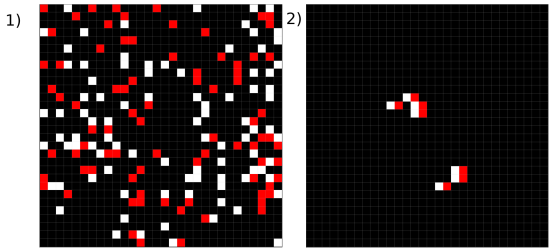

Figure 7. 1) a random initial condition for the automation 2) system after evolving into a stable state with two infinitely moving gliders.

By experimenting, several initial conditions were constructed that would lead to interesting developments of the system such as a periodic oscillator or a pattern that infinitely generates more moving patterns. Some interesting initial conditions are pictured below.

First a number of infinitely moving gliders were identified. Those kinds of patterns move infinitely in the initial direction unless they collide with another pattern. Since the grid boundary conditions are periodical the infinite movement is represented by periodically crossing one boundary and appearing on another. On figure 8 are brought three such examples. On this and other figures black indicates the ready state, red indicates the firing state and white indicates the resting state.

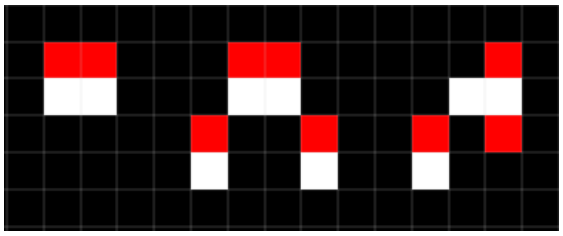

Figure 8. Three examples of infinitely moving patterns. Two patterns on the left move in the upward direction and the one on the right moves diagonally to right-left.

By having two simple gliders collide with eachother using initial conditions shown in figure 9 it is possible to create two new gliders that move at a right angle to the initial motion giving the impression of two bouncing gliders.

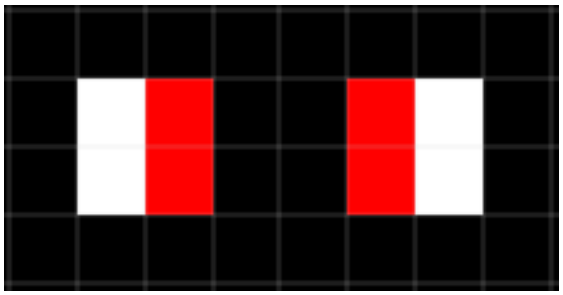

Figure 9. Initial condition that leads to two gliders bouncing in a 90 degree angle

It is also possible to set up a pattern that stais in the same location and oscillates infinitely. An oscillator with three distinct steps was identified that is pictured in figure 10

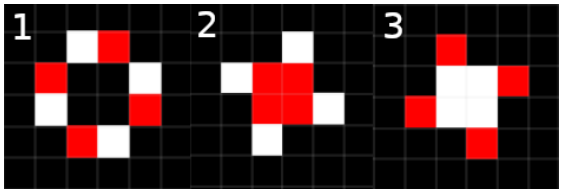

Figure 10. A three step oscillating pattern

On figure 11 is pictured a pattern that moves forward and periodically creates more moving patterns.

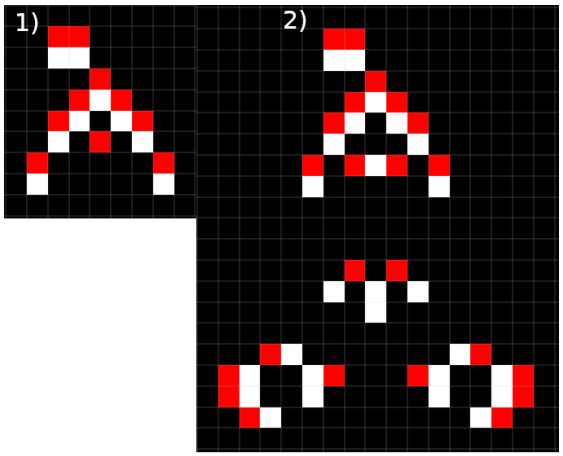

Figure 11. A pattern that while moving forward creates other moving patterns. 1) initial condition 2) system after some steps. the bottom two shapes are gliders moving to the left and right.

4. Greenberg-Hastings Cellular Automaton

4.1. GHCA theory

GHCA THEORY HERE

4.2. GHCA Method

FOR IMPLEMENTATION OF GHCA...

4.3. GHCA Results

FOR RESULTS OF GHCA...

5. Interpretation

INTERPRETATION GOES HERE.

6. Conclusion

In Section (2.5) we presented snapshots illustrating the realization of logic gates within a computer simulation of the Game of Life. From this, one may conclude that a Boolean basis is effectively implemented within the system. Once the principles for composing these gates into larger circuits are understood, increasingly complex computational structures can be constructed.

The existence of logic gates in the Game of Life implies that they form a functionally complete basis for Boolean logic. Consequently, any Boolean function can be expressed as a finite composition of such gates. This property is fundamental for the construction of general computational devices. In particular, it is possible to design patterns that behave as a finite-state machine coupled to two counters. Such a system is computationally equivalent to a universal Turing machine

It follows that the Game of Life possesses the same theoretical computational power as any computer with unbounded memory and no time constraints; that is, it is Turing complete [5]. This demonstrates that complex dynamics can emerge solely from prescribed initial conditions and a simple update rule, forming a discrete dynamical system.

7. Use of LLM tools

7.1. Game of Life

RLE prompt: Create an rle loader to import standard Game of Life patterns directly. The output was a complete functional script `RLE.py` that parses strings like

```
24bo11b$22bobo11b$12b2o6b2...
```

, which encode binary patterns stored in a Wiki page.

Text Files prompt: Generate three text files containing the initial live-cell coordinates for an AND gate, an OR gate, and a NOT gate. The files should be called `AND.txt`, `OR.txt`, and `NOT.txt`. Each file should contain one live cell per line, written in the format `(x,y)` This was used to generate possible text files, so one can try running them with the code.

Debug interface prompt: Take the simple visualization of the precomputed Game of Life and add functionality so that space bar works as pause/play and left/right as previous/next frames and output the python file `GameOfLifeDebug.py`. This was used for quick prototyping of the engineering of the logic gates.

7.2. Neuron Cellular Automaton

The code in `neuron.py` is based on `gameOfLifeInteractiveToric.py` from the lecture. We used ChatGPT to modify the code. The prompt was ”I would like to change the rules of the game. I need to have three different states for each cell now and new rules, how can I implement this?” and additionally added the original code. This returned good instructions on where the code needs modification. Additionally, we looked up the documentation of the functions used by ChatGPT and made some modifications to its suggestions.

Files submitted

The following files have been submitted with this report:

1. `GameOfLife.py`
2. `GameOfLifeDebug.py`
3. `GoLogic.py`
4. `pattern_processing.py`
5. `RLE.py`
6. `NOT.txt`
7. `AND.txt`
8. `OR.txt`

References

[1] Alex Bellos. Game of life: Logic gates.

[2] Jordi-Lluís Figueras. Interactive conway's game of life board for classroom demonstrations, 2026.

[3] LifeWiki. Glider.

[4] LifeWiki. Glider gun.

[5] Wikipedia. Turing completeness.